Algoritmos y Estructuras de Datos

Ejercicios resueltos en la clase del 29 de octubre
Elección de Estructuras
Segundo Cuatrimestre 2026

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Enunciados

Ejercicio 10: Maderera San Blas

La Maderera San Blas vende, entre otras cosas, listones de madera. Los compra en aserraderos de la zona, los cepilla y acondiciona, y los vende por menor del largo que el cliente necesite.

Tienen un sistema un poco particular y ciertamente no muy eficiente: Cuando ingresa un pedido, buscan el listón más largo que tiene en el depósito, realizan el corte del tamaño que el cliente pidió, y devuelven el trozo que queda al depósito.

Por otra parte, identifican a cada cliente con un código alfanumérico de 10 dígitos y cuentan con un fichero en el que registran todas las compras que hizo cada cliente (con la fecha de la compra y el tamaño del listón vendido).

Este sería el TAD simplificado del sistema:

```
   Cliente es string
TAD Maderera {
   proc comprarUnListon(inout m : Maderera, in tamaño : ℤ) {
      // comprar en el aserradero un listón de un determinado tamaño
   }
   proc venderUnListon(inout m : Maderera, in tamaño : ℤ, in cli : Cliente, in f : Fecha) {
      // vender un listón de un determinado tamaño a un cliente particular en una fecha determinada
   }
   proc ventasACliente(in m : Maderera, in cli : Cliente) {
      // devolver el conjunto de todas las ventas que se le hicieron a un cliente
      // (para cada venta, se quiere saber la fecha y el tamaño del listón)
   }
}
```

Se pide:

a) Escriba una estructura que permita realizar las operaciones indicadas con las siguientes complejidades:

- comprarUnListon en $O(\log(m))$
- venderUnListon en $O(\log(m))$
- ventasACliente en $O(1)$

donde m es la cantidad de pedazos de listón que hay en el depósito

b) Escriba el algoritmo para la operación `venderUnListon`

Agenda (Parcial 2023)

Se quiere implementar el `TAD Agenda` que modela una agenda semanal donde se registran actividades. Cada actividad tiene un identificador, un horario de inicio y uno de finalización. No puede haber dos actividades con el mismo identificador. Para simplificar, las actividades sólo pueden comenzar y terminar en horarios en punto (por ejemplo, 21:00 hs) y terminan en un horario posterior a su inicio. Tampoco pueden empezar y terminar en días diferentes. Para contar la cantidad de actividades que transcurren en un determinado horario no se debe tener en cuenta aquellas que finalizan en ese momento. En cada actividad se pueden agregar tags que permiten agrupar las distintas actividades por temáticas. Los tags tienen cómo máximo 20 caracteres.

Dadas las siguientes operaciones y de acuerdo a las complejidades temporales de peor caso indicadas, donde d es la cantidad de días registrados hasta el momento y a la cantidad total de actividades, respectivamente:

- `proc RegistrarActividad`(inout $ag : Agenda$, in $act : IdActividad$, in $dia : Dia$, in $inicio : Hora$, in $fin : Hora$)
  - **Requiere**: La hora de fin sea mayor que la de inicio y la actividad no está actualmente registrada.
  - **Descripción**: Se agrega la actividad a la agenda, marcando en el dia indicado la hora de inicio y finalización.
  - **Complejidad**: $O(log(a) + log(d))$
- `proc VerActividad`(in $ag : Agenda$, in $act : IdActividad$) : struct $\langle dia : Dia, inicio : Hora, fin : Hora \rangle$
  - **Requiere**: La actividad debe estar registrada en la agenda.
  - **Descripción**: Se devuelven el dia y horario en que se realiza la actividad.
  - **Complejidad**: $O(log(a))$.
- `proc AgregarTag`(inout $ag : Agenda$, in $act : IdActividad$, in $t : Tag$)
  - **Requiere**: La actividad está registrada en la agenda y aún no tiene registrado ese tag.
  - **Descripción**: Se agrega el tag a la actividad indicada.
  - **Complejidad**: $O(1)$.
- `proc HoraMásOcupada`(in $ag : Agenda$, in $d : Dia$) : $Hora$
  - **Requiere**: El dia indicado tiene al menos una actividad registrada.
  - **Descripción**: Se devuelve la hora que tiene más actividades registradas.
  - **Complejidad**: $O(log(d))$.
- `proc ActividadesPorTag`(in $ag : Agenda$, in $t : Tag$) : Conjunto$\langle IdActividad \rangle$
  - **Requiere**: Verdadero.
  - **Descripción**: Se devuelven las actividades que tienen registrado el tag t.
  - **Complejidad**: $O(1)$.

Se pide:

a) Definir la estructura de representación del módulo `AgendaImpl`, que provea las operaciones solicitadas. Se debe explicar detalladamente qué información se guarda en cada parte y las relaciones entre ellas.

b) Explicar cómo se cumplen las complejidades pedidas por cada operación. Indicar todas las suposiciones tomadas sobre la implementación de las estructuras mencionadas.

c) Escribir el algoritmo de `RegistrarActividad` justificando **detalladamente** que se cumple la cota de de complejidad requerida.

Seguimiento de Tesis (Parcial 1c 2025)

Desde el área de coordinación académica de una universidad nos piden desarrollar un sistema que permita hacer un seguimiento de las tesis de grado de los estudiantes. Nos piden ayuda para organizar la información de las carreras y los alumnos que presentan tesis.

Los alumnos pueden ser identificados con un número entero positivo de hasta 8 dígitos. Cada alumno pertenece a una carrera y puede presentar una única tesis con una fecha estimada de defensa (número de día del calendario académico, acotado entre 1 y 365). Además, se quiere registrar la calificación obtenida al momento de defender la tesis. Las tesis, por otro lado, se pueden identificar por un identificador que combina el ID del estudiante, la carrera y otros valores aleatorios. De momento no tiene ninguna restricción de longitud.

El sistema deberá implementar las siguientes operaciones en la complejidad pedida, donde c es la cantidad de carreras registradas, a la cantidad de alumnos, t la cantidad de tesis. $Carrera$, $Alumno$, $Fecha$ y $Tesis$ son $\mathbb{Z}$.

- `proc InicializarSistema`() : $Sistema$
  - **Descripción**: Inicializa el sistema sin carreras, alumnos ni tesis.
  - **Complejidad**: $O(1)$
- `proc RegistrarAlumnos`(inout $s : Sistema$, in $c : Carrera$, in as : Conjunto$\langle Alumno \rangle$)
  - **Descripción**: Registra los alumnos en la carrera indicada.
  - **Requiere**: Hay al menos un alumno en as y ninguno pertenece a otra carrera.
  - **Complejidad**: $O(\log(c) + a)$
- `proc AsignarTesis`(inout $s : Sistema$, in $a : Alumno$, in $t : Tesis$, in $f : Fecha$) : bool
  - **Descripción**: Si el alumno no tiene una tesis registrada, le asigna la tesis t con fecha estimada de defensa en f. En res se devuelve si se pudo registrar la tesis.
  - **Requiere**: El alumno está registrado en alguna carrera.
  - **Complejidad**: $O(\log(t))$
- `proc DefenderTesis`(inout $s : Sistema$, in $a : Alumno$, in $nota : \mathbb{Z}$)
  - **Descripción**: Registra que el alumno defendió su tesis con la nota recibida.
  - **Complejidad**: $O(1)$
- `proc TesisRegistradasEn`(in $s : Sistema$, in $f : Fecha$) : Conjunto$\langle Tesis \rangle$
  - **Descripción**: Devuelve las tesis agendadas para la fecha f.
  - **Complejidad**: $O(1)$
- `proc KTesisMasAntiguas`(in $s : Sistema$, in $k : \mathbb{Z}$) : Conjunto$\langle Tesis \rangle$
  - **Descripción**: Devuelve las k tesis más antiguas del sistema.
  - **Requiere**: k es a lo sumo t.
  - **Complejidad**: $O(k \cdot \log(t))$

a) Plantear la estructura de representación del módulo `Sistema` que permita cumplir con las complejidades requeridas. **Justificar** con palabras de forma concisa las operaciones **InicializarSistema**, **RegistrarAlumnos**, **AsignarTesis**, **DefenderTesis** y **TesisRegistradasEn**.

b) Escribir el algoritmo de la operación `KTesisMasAntiguas`. Justifique **detalladamente** que se cumple la complejidad obtenida.

Soluciones

Maderera San Blas

Para implementar un TAD cumpliendo complejidades que nos indican un buen método es, primero entender qué información necesitamos guardar (generalmente va a ser la misma que está en los observadores del TAD) y después recorriendo las operaciones y tratando de entender qué restricciones nos están imponiendo a cómo guardar esta información.

En el caso de la maderera vamos a tener que guardar dos cosas:

- Los listones a vender
- Los clientes junto con la información de cada compra que hizo (que desde el punto de la maderera se llaman ventas)

Las operaciones que nos piden son las siguientes, desglosadas en todo lo que hace cada operación sobre nuestra estructura:

- comprarUnListon en $O(\log(m))$
  - Agrega un listón de un tamaño L a la colección de listones
- venderUnListon en $O(\log(m))$
  - Saca el listón más largo del depósito
  - Guarda el listón sobrante si corresponde
  - Registra la venta
- ventasACliente en $O(1)$
  - Devuelve todas las ventas hechas al cliente

Vemos que la complejidad de nuestras operaciones sólo depende de la cantidad de listones existentes y no depende ni de la cantidad de clientes registrados ni de las ventas que se hicieron. Sin embargo la cantidad de clientes es arbitraria, debería ser imposible tener una complejidad constante para buscar la información de uno de ellos. Por suerte tenemos un dato más: Los clientes se identifican con un código alfanumérico de tamaño fijo. Esto nos da la pista de que podemos implementar el fichero con un diccionario digital (o diccionario sobre *Trie*) de clientes que tiene el conjunto de ventas como significado, y obtener los datos en $O(1)$ como nos piden.

¿Cómo resolvemos las otras operaciones? Para guardar los listones en $O(log(m))$ podemos usar un conjunto logarítmico (sobre AVL) ordenando los listones por algún criterio, por ejemplo por tamaño. De hecho para representar los listones lo único que nos importa es el tamaño así que podemos representarlos directamente como `int`, y guardarlos en un conjunto.

Con estos dos criterios tendríamos esta primera versión del Módulo

```
  Liston es int
Cliente es string
Módulo MadereraImpl implementa Maderera <

  var listones: ConjLog<Liston>
  var clientes: DiccDigital<Cliente, Conjunto<Tupla<Fecha, Liston>>>

>
```

Y podemos revisar si cumple las complejidades pedidas:

- comprarUnListon en $O(\log(m))$
  - Agrega un listón de un tamaño L al conjunto logarítimico de listones en $O(log(m))$ ✓
- venderUnListon en $O(\log(m))$
  - Saca el listón más largo del depósito – Buscarlo entre todos los elementos en $O(m)$ ✗
  - Guarda el listón sobrante si corresponde en $O(log(m))$ ✓
  - Registra la venta – Esto debería ser $O(1)$. Acceder a las ventas del cliente es $O(1)$, pero necesitamos que agregar la venta también lo sea ✗
- ventasACliente en $O(1)$
  - Devuelve todas las ventas hechas al cliente en $O(1)$ ✓

Tenemos dos problemas a resolver. Por un lado agregar las ventas al conjunto de cada cliente. Podemos asumir que todas las ventas son diferentes, entonces podemos usar la operación **agregarRapido**, que es $O(1)$ si usamos un Conjunto Lineal, es decir, sobre una lista enlazada.

Para resolver el problema de obtener el listón más grande, podemos usar una cola de prioridad, que nos permite:

- Obtener el valor máximo en $O(1)$
- Desencolarlo en $O(log(m))$
- Encolar uno nuevo en $O(log(m))$

La nueva estructura sería

```
  Liston es int
Cliente es string
Módulo MadereraImpl implementa Maderera <
  var listones: ColaPrioridadMax<Liston>
  var clientes: DiccTrie<Cliente, ConjLineal<Tupla<Fecha, Liston>>>
>
```

Y ahora

- comprarUnListon en $O(\log(m))$
  - Agrega un listón de un tamaño L al conjunto logarítimico de listones en $O(log(m))$ ✓
- venderUnListon en $O(\log(m))$
  - Saca el listón más largo del depósito – Buscarlo entre todos los elementos en $O(m)$ ✓
  - Guarda el listón sobrante si corresponde en $O(log(m))$ ✓
  - Registra la venta – Acceder a las ventas del cliente es $O(1)$ y agregar la nueva también ✓
- ventasACliente en $O(1)$
  - Devuelve todas las ventas hechas al cliente en $O(1)$ ✓

Ahora sólo falta escribir los algoritmos

```
function CompraRUnListon(inout M: Maderera, in l: Liston)
    res := M.listones.encolar(l)                                   ▷ O(log(m))
end function
function VenderUnListon(inout M: Maderera, in tamaño: Liston, in c: Cliente, in f: Fecha)
    int maxTamañoDisponible := M.listones.desencolar()             ▷ O(log(m))
    int remanente := maxTamañoDisponible - tamaño                  ▷ O(1)
    if remanente > 0 then
        M.listones.encolar(remanente)                              ▷ O(log(m))
    end if
    Tupla<Fecha, Liston> nuevaVenta := <f, tamaño>
    if ! M.clientes.está(c) then                                   ▷ O(1)
        M.clientes.definir(c, new ConjLineal<Tupla<Fecha, Liston>>) ▷ O(1)
    end if
    ConjLineal<Tupla<Fecha, Liston>> ventasCliente := M.clientes.obtener(c) ▷ O(1)
    ventasCliente.agregarRapido(nuevaVenta)                        ▷ O(1)
end function
function VentasAClienteinout M: Maderera, in c: Cliente)Conjunto<Tupla<Fecha, Liston>>
    res := M.clientes.obtener(c)                                   ▷ O(1)
end function
```

Agenda

Dia, Hora, IdActividad `ES` int
Tag `ES` string

Módulo AgendaImpl **implementa** Agenda `<`

```
/* Para cada actividad, la correspondiente fecha de inicio y fin */
var actividades: DiccionarioLog<IdActividad, Struct<dia: Dia, inicio: Hora, Fin: Hora>>

/* Para cada tag, las actividades que corresponden */
var tags: DiccionarioDigital<Tag, ConjuntoLineal<IdActividad>>

/* Para cada día, un vector de 24 posiciones (una por hora) */
/*  con el conjunto de actividades agendadas para esa hora    */
var dias: DiccionarioLog<Dia, Vector<ConjuntoLineal<IdActividad>>>
```

`>`

Informalmente el invariante de representación será:

- Todos los arreglos que son valores de `dias` tienen exactamente 24 posiciones.
- Todas las claves de `actividades` están en los conjuntos de los vectores de `dias`, en vector que corresponde al día que aparece en el struct significado `actividades`, y en las posiciones que corresponden a las horas.
- Todos los *idActividad* de todos los conjuntos de `tags` son claves de `actividades`.
- Para todos los días que son clave de `dias` hay algún struct significado en `actividades` que tiene ese día en la primer posición
- Además, para todos esos días, todos los *IdActividad* en los conjuntos que están en los vectores significado son clave de `actividades`

Observación: Además de que no lo pide el enunciado, no se puede definir la función de abstracción ya que desconocemos los observadores del TAD.

Veamos cómo se cumplen las complejidades:

- **RegistrarActividad**:
  - Se agrega la actividad al diccionario `actividades` en $O(log(a))$.
  - En cuanto al diccionario `dias`
    - **Si es la primer actividad del día** se agrega el día al diccionario en $O(log(d))$.
    - **Si el día ya está definido** se lo busca en el diccionario para obtener el vector de horas, también en $O(log(d))$
    - Se agregan las actividades en las posiciones del vector correspondiente. Las horas posibles están acotadas (24) y como la actividad es nueva, podemos usar *agregarRápido* en los conjuntos, que por ser lineales es $O(1)$.
    - La complejidad total de esta sub-operacion es $O(log(d) + O(1) + t \times O(1)) = O(log(d))$ donde t es la cantidad de horas que dura la actividad.
  - Total: ***O(log(a) + log(d))***.
- **VerActvidad**: Se busca la actividad en el diccionario `actividades`, en ***O(log(a))***.
- **AgregarTag**:
  - **Si el tag no existe** se lo define en el diccionario `tags` en $O(1)$ por ser un diccionario digital (sobre trie) con *strings* de largo acotado, con un conjunto lineal que tiene únicamente la actividad (es decir, se construye también en $O(1)$)
  - **Si el tag existe** se busca el conjunto de actividades que corresponde al tag en $O(1)$ (por lo antedicho). Por el requiere del proc, sabemos que la actividad no está en el conjunto, por lo que podemos usar la operación *agregarRápido*, que en un conjunto líneal es $O(1)$.
  - En ambos casos, el total es ***O(1)***.
- **HoraMasOcupada**:
  - Se busca el día en cuestión en el diccionario `dias` en $O(log(d))$.
  - Se recorre todo el vector hasta encontrar la hora con más actividades. Como el vector tiene tamaño fijo 24, esta operación es $O(1)$.
  - Total: ***O(log(d))***.
- **ActividadesPorTag**: Se busca el conjunto de actividades en el diccionario `tags`. Por las razones que ya dijimos, esta operación es ***O(1)***.

Algoritmos

El enunciado sólo pide el de **RegistrarActividad**, presentamos todas para ejemplificar

```
proc Crear(): AgendaImpl
  // Creo todas las variables con sus tipos
  // Todas las operaciones son O(1)
  AgendaImpl res := new AgendaImpl()

  res.actividades :=
    new DiccionarioLog<IdActividad, Struct<dia: Dia, inicio: Hora, Fin: Hora>>()
  res.tags := new DiccionarioDigital<Tag, ConjuntoLineal<IdActividad>>()
  res.dias := new DiccionarioLog<Dia, Vector<ConjuntoLineal<IdActividad>>>()

  return res
endproc


proc RegistrarActividad(
    inout ag: AgendaImpl, in act: IdActividad,
    in Dia: Dia, in Inicio: Hora, in fin: hora
  )
  // Agrego la actividad, que por requiere no está registrada previamente
  // Esta operación es O(log(a))
  ag.actividades.definir(act, new struct<dia: Dia, inicio: Inicio, fin: Fin>())

  // Ahora tengo que agregarla a los días
  // Como se que no está registrada, puedo agregarRapido en O(1)
  // Pero si el día no está registrado hay que registrarlo en O(log(d))
  if ! ag.dias.esta(dia)
    Vector<ConjuntoLineal> horarios := new Vector(24)

    hora i := inicio
    while (i < fin)
      horarios[i].agregarRapido(act)
      i++
    endwhile

    ag.dias.definir(dia, horarios)
  else
    hora i := inicio
    while (i < fin)
      ag.dias.obtener(dia)[i].agregarRapido(act)
      i++
    endwhile
  endif
endproc

proc VerActividad(in ag: AgendaImpl, in act: IdActividad) :
      struct <dia: Dia, inicio: Hora, fin: Hora>
  // Solo tengo que obtener la actividad del DiccionarioLog y devolverla
  return ag.Actividades.obtener(act)
endproc

proc AgregarTag(inout ag: AgendaImpl, in act: IdActividad, in t: Tag)
  // Por requiere sabemos que la actividad está registrada y que no tiene este Tag
  if ! ag.tags.esta(t)
    ConjuntoLineal<IdActividad> actividades := new ConjuntoLineal()
    actividades.agregarRapido(act)
    ag.tags.definir(t, actividades)
  else
    ag.tags.obtener(t).agregarRapido(act)
  endif
endproc
```

```
proc HoraMasOcupada(in ag: AgendaImpl, in d: Dia): Hora
  h := 0
  max := 0
  res := 0
  while (h < 24)
    // Obtener el día en el DiccionarioLog es O(log(n))
    actividades\_h := ag.dias.obtener(d)[h]
    if (actividades\_h.length > max)
      max := actividades\_h
      res := h
    endif
  endwhile
  return res
endproc

proc ActividadesPorTag(in ag: AgendaImpl, in t: Tag): Conjunto<IdActividad>
  res := ag.tags.obtener(t)
  return res
endproc
```

Tesis

Estructura

Módulo SistemaImpl **implementa** Sistema <

```
var alumnosPorCarrera: DiccionarioLog<Carrera, ConjuntoTrie<Alumno>>
var tesisPorAlumno: DiccionarioTrie<Alumno, Tesis>
var notaDeTesis: DiccionarioTrie<Alumno, int>
var tesisPorFecha: DiccionarioTrie<Fecha, ConjuntoLog<Tesis>>
/* Ordenando por fecha */
var tesisMasAntiguas: ColaDePrioridadLog<struct<tesis: Tesis, fecha: Fecha>>
```

>

Las operaciones se van a usar de la siguiente manera:

- **IniciarSistema**:
  - Se definen todos los Diccionarios y la ColaDePrioridad vacíos en $O(1)$.
  - Para `tesisPorFecha` se definen las 365 claves con Conjuntos vacíos como significados, como definir cada fecha tiene costo $O(1)$ por ser un trie con claves acotadas, el costo total es $O(365) = O(1)$.
  - Total: ***O(1)***
- **RegistrarAlumnos**:
  - Buscamos la carrera en `alumnosPorCarrera` con costo $O(log(c))$.
  - Creamos un conjunto vacío y vamos definiendo uno a uno los alumnos de la carrera verificando previamente si ya no están registrados. Como es un conjunto trie, cada búsqueda y definición es $O(1)$ por lo que el total es $O(a)$
  - Total: ***O(log(c) + a)***
- **AsignarTesis**:
  - Buscamos al alumno en `tesisPorAlumno` y definimos la tesis registrada. Como usamos un DiccTrie y los ID están acotados esto tiene costo $O(1)$.
  - Buscamos la fecha que recibimos en `tesisPorFecha` ($O(1)$ porque las fechas están acotadas) y agregar la tesis al conjunto correspondiente ($O(log(t))$).
  - Agregamos la tesis a `tesisMasRecientes`, también en $O(log(t))$.
  - Total: $O(1) + O(1 + log(t)) + O(log(t)) =$ ***O(log(t))***.
- **TesisPorFecha**: Como las días están acotados entre 1 y 365, acceder a cualquier clave en `tesisPorFecha` tiene costo ***O(1)***.
- **DefenderTesis** Ingresamos a `notaDeTesis` con la clave del alumno y definimos la nota obtenida. Como es un DiccTrie también esto tiene costo ***O(1)***.

```
proc KTesisMasAntiguas(in s: Sistema, in k: int): Conjunto<Tesis>
  resultado := ConjuntoVacio();                    // O(1)
  temporales := ListaVacia();                      // O(1)
  i := 0;                                          // O(1)

  // Busco los elementos a devolver
  while (i < k)                                    // O(k)
    if (s.tesisMasRecientes.Vacia())               // O(1)
      break;                                       // O(1)
    endif

      par := s.tesisMasRecientes.Desencolar()      // O(log(t))
      resultado.Agregar(par.tesis)                 // O(log(k))
      temporales.Agregar(par)                      // O(1)
      i := i + 1                                   // O(1)
  endwhile

  // Recompongo la estructura
  i := 0
  while (i < temporales.longitud())                // O(k)
      s.tesisMasRecientes.Encolar(par)             // O(log(t))
  endwhile

  return resultado                                 // O(1)
endproc
```

Complejidad total: $O\big(3 + k.(4 + \underbrace{log(t) + log(k)}_{\text{por definición } k \leq t}) + k.log(t)\big) = O(k.log(t) + k.log(t)) = \mathbf{O(k.log(t))}$